گزارش اجرای کار

گزارش اجرای سه مرحله‌ی سئو، بهینه‌سازی سرعت و امنیت

پلتفرم کارت ویزیت دیجیتال آلوشمال

تاریخ: ۲۵ مهر ۱۴۰۵

نسخه‌ی نهایی پروژه: `v2.53.0`

مستندات مرتبط: `History-Agent.md` ، `docs/SECURITY-AUDIT.md` ، `docs/REVENUE-MODELS.md`

خلاصه‌ی مدیریتی

در این چرخه‌ی کاری، سه مرحله‌ی مستندات راهنمای پروژه به‌ترتیب اجرا شد: **مرحله‌ی یک (سئو)** بر پایه‌ی `seocart-1 .pdf` و `seocart-2.pdf` ، **مرحله‌ی دو (سرعت)** بر پایه‌ی `speed.pdf` و `speed-2.pdf` ، و **مرحله‌ی سه (امنیت)** بر پایه‌ی `secure.pdf` . هر مرحله پس از پیاده‌سازی تست و در یک کامیت مستقل ثبت شد؛ نسخه‌ی پروژه از `v2.49.0` به `v2.53.0` رسید.

مرحله	موضوع	وضعیت	نسخه	کامیت
–	مستندسازی چهار مدل کسب درآمد	انجام‌شده	`v2.50.0`	`930ec8b`
یک	سئو و رندر دایرکتوری‌ها	انجام‌شده	`v2.51.0`	`a6727fe`
دو	بهینه‌سازی سرعت	انجام‌شده	`v2.52.0`	`e4ab7ca`
سه	سخت‌سازی امنیت	انجام‌شده	`v2.53.0`	`eb8f55f`

۱. مستندسازی چهار مدل کسب درآمد

مستند `docs/REVENUE-MODELS.md` به‌عنوان مرجع فنیِ برنامه‌نویس نوشته شد و در `README` به آن ارجاع داده شد. این سند چهار مدل درآمدی پلتفرم و یک جریان مکمل را با جزئیات پیاده‌سازی توضیح می‌دهد:

#	مدل	معنی	برچسب	مدت‌دار؟
۱	خرید اشتراک	فروش سهمیه‌ی تعداد آیتم و سطح نمایش فیلدها (سقف ساخت)	–	بله (پلن)
۲	پله	بردن آیتم به صدر **دسته‌ی همان شهر + کسب‌وکار**، انگار تازه ساخته شده	«پله»	بله
۳	تبلیغ در دایرکتوری	تبلیغ **هدفمند**: فقط وقتی بازدیدکننده همان شهر و همان کسب‌وکار را فیلتر کند، آیتم اول فهرست می‌آید	«تبلیغات»	بله
۴	تبلیغ در صفحه‌ی اصلی	نمایش آیتم در جعبه‌ی تبلیغات صفحه‌ی خانه به مدت تعیین‌شده‌ی ادمین	«تبلیغات»	بله
۵	اشتراک فید (مکمل)	فروش بسته‌ی سهمیه‌ی پست تصویری برای فیدها	–	بله

> **تفکیک حیاتی:** «پله» رایگان/سهمیه‌ای است و برچسبش «پله» است؛ «تبلیغ در دایرکتوری» پرداختی، مدت‌دار، برچسبش «تبلیغات» و اولویت بالاتری از پله دارد. این دو هرگز نباید با هم اشتباه شوند.

۲. مرحله‌ی یک — سئو و رندر دایرکتوری‌ها

بر پایه‌ی `seocart-1 .pdf` (خزش‌نشدن اسکرول بی‌پایان و قطع زنجیره‌ی لینک‌سازی داخلی) و `seocart-2.pdf` (ساختار سئوی کارت‌های ویزیت با اسلاگ عددی و JSON-LD).

تغییرات

- **ماژول سئو در API**: ماژول جدید `apps/api/src/seo/` با اندپوینت عمومی `GET /seo/sitemap` که کارت‌های ویزیت، بیولینک‌ها و پست‌های فیدِ منتشرشده و تأییدشده به‌همراه صفحات ثابت را برمی‌گرداند (۲۵۱۷ ورودی).
- **سایت‌مپ و robots**: `app/sitemap.ts` (با revalidate ساعتی)، `app/robots.ts` و آیکون `app/icon.svg` .
- **متادیتا و JSON-LD**: در صفحات جزئیات کارت ویزیت و بیولینک، `generateMetadata` با canonical و Open Graph/Twitter اضافه شد و داده‌ی ساخت‌یافته (LocalBusiness برای کسب‌وکار، ProfilePage برای بیولینک) تزریق شد؛ صفحات ناموجود `noindex` شدند.
- **رندر سمت سرور دایرکتوری‌ها**: فایل `lib/directory-server.ts` و گذار صفحات دایرکتوری به Server Component؛ منطق تعاملی به کامپوننت‌های جدید `DirectoryBiolinkContent.tsx` و `DirectoryBusinessCardContent.tsx` منتقل شد. اکنون ردیف‌ها لینکِ واقعی `<a href>` با متن توصیفی دارند تا خزنده‌ی گوگل آن‌ها را ببیند.

> **تأیید:** در HTML رندرشده‌ی سمت سرور لینک‌های قابل‌خزش وجود دارد؛ `robots.txt` ، `sitemap.xml` و `icon.svg` کد ۲۰۰ برمی‌گردانند؛ عنوان/canonical/JSON-LD روی صفحات جزئیات حاضرند؛ اسکرول بی‌پایان (۲۰ → ۷۰ ردیف) و نمای دوقسمتی کار می‌کنند.

۳. مرحله‌ی دو — بهینه‌سازی سرعت

بر پایه‌ی `speed.pdf` (تحلیل ترافیک شبکه و گلوله‌های N+1) و `speed-2.pdf` (اقدامات فوری بدون ویرایش کد، از جمله تست بار با Autocannon).

تغییرات

- **حذف درخواست N+1 لایک**: به `LikeButton` پراپ `reconcile` اضافه شد؛ ردیف‌های دایرکتوری با `reconcile={false}` دیگر درخواست مستقل `GET .../like` نمی‌فرستند (۵ درخواست → صفر). یک افکت آینه‌ای با `useRef` نیز حالت واقعی والد را بدون پاک‌کردن کلیک خوش‌بینانه‌ی کاربر همگام می‌کند.
- **فروکش پیش‌واکشی ناوبری**: به لینک‌های `Footer` ، `MobileNav` و `UserShell` پراپ `prefetch={false}` اضافه شد تا سیل درخواست `?_rsc=` روی مسیرهای سنگین (پنل و ادمین) متوقف شود.

> **تأیید:** `tsc` تمیز؛ صفحه‌ی ۵ ردیفی صفر درخواست لایک می‌فرستد؛ تست بار Autocannon روی `GET /business-cards/public`: میانه‌ی ~۴۸ms، میانگین ~۵۱ms، توان ~۳۸۸ درخواست/ثانیه، بدون پاسخ غیر-2xx.

۴. مرحله‌ی سه — سخت‌سازی امنیت

بر پایه‌ی `secure.pdf` (چک‌لیست شش‌لایه‌ی امنیت پروژه).

تغییرات

- **هدرهای امنیتی و CSP فرانت‌اند:** فایل `apps/web/proxy.ts` (جایگزین `middleware.ts` بر پایه‌ی قرارداد جدید Next.js 16) روی همه‌ی مسیرها هدرهای `X-Frame-Options: DENY`، `X-Content-Type-Options`، `Content-Security-Policy`، `Cross-Origin-Opener-Policy`، `Permissions-Policy`، `Referrer-Policy` و در `Strict-Transport-Security` را HTTPS اضافه می‌کند.
- **کاربر غیرروت API:** در مرحله‌ی production فایل `apps/api/Dockerfile` کاربر `node` تنظیم شد (به‌همراه ساخت و chown پوشه‌ی `backups` برای اسنپ‌شات entrypoint).
- **اسکن خودکار وابستگی‌ها:** فایل `.github/dependabot.yml` برای اکوسیستم‌های `github-actions`، `npm` و `docker` فعال شد.
- **سند ممیزی:** `docs/SECURITY-AUDIT.md` چک‌لیست را لایه‌به‌لایه (SAST/Secret، SCA، بک‌اند Prisma/NestJS، فرانت‌اند، DAST، داکر/Postgres) مرور و وضعیت هر مورد را در مخزن ثبت می‌کند.

نتیجه‌ی بررسی چک‌لیست

لایه	مورد	وضعیت در مخزن
SCA	اسکن خودکار وابستگی‌ها	پیاده‌شده Dependabot فعال شد
SAST/Secret	Semgrep / gitleaks	رویه‌ی آماده دستورها در سند ممیزی ثبت شد
بک‌اند	SQL Injection / `$queryRawUnsafe`	امن فقط `$queryRaw` در health check استفاده شده
بک‌اند	اعتبارسنجی ورودی (Mass Assignment)	فعال `ValidationPipe` سراسری با whitelist
بک‌اند	هدرها، CORS و Rate Limiting	فعال helmet، CORS محدود، محدودیت نرخ روی Redis
فرانت‌اند	CSP و هدرهای امنیتی	پیاده‌شده `proxy.ts`
فرانت‌اند	XSS / متغیرهای محیطی	امن تنها JSON-LD تزریق می‌شود؛ هیچ Secretی با `NEXT_PUBLIC_` نیست
DAST	OWASP ZAP	رویه‌ی آماده دستور در سند ممیزی
داکر/DB	عدم اجرای Root و عدم اکسپوز 5432	پیاده‌شده `USER node/nextjs` + پورت‌های پروداکشن محدود به `127.0.0.1`

> **تأیید:** `tsc` بدون خطا؛ `curl -D -` روی مسیرهای `/dashboard` ، `/biolinks` ، `/` و `/api/health` حضور همهٔ هدرهای امنیتی را نشان داد و هشدار deprecated صادر نشد؛ `sync_install.py --check` → بدون تغییر معلق.

۵. کنترل کیفیت و قواعد پروژه

- هر تغییر پس از تست، در `History-Agent.md` ثبت و نسخه در `apps/web/lib/version.ts` بالا برده شد (نسخه‌ی نهایی `v2.53.0`).
- پس از هر ویرایش، `install.sh` با `python3 scripts/sync_install.py` همگام و با `--check` تأیید شد (بدون تغییر معلق).
- هر مرحله در یک کامیت مستقل و با پیام معنادار ثبت شد؛ `push` طبق قواعد پروژه بر عهده‌ی کاربر است.

۶. گام‌های بعدی پیشنهادی

- راه‌اندازی اجرای دوره‌ای `npm audit` و `gitleaks` در CI (فراتر از Dependabot) و یک اسکن پایه‌ی OWASP ZAP روی اندپوینت‌ها.
- جایگزینی `'unsafe-inline'` در CSP با یک خط‌لوله‌ی nonce برای سخت‌ترکردن بیشتر سیاست امنیت محتوا.
- افزودن اسکن ایمیج با `trivy` به‌عنوان یک گام در گردش کار انتشار.
- پیگیری موارد معوق ثبت‌شده در `BACKLOG.md` .